Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Seguridad de la Información • CE-1115

Taller de Criptografía Aplicada

Chat cifrado entre dos procesos locales con criptografía híbrida

SecureChat: descripción del protocolo

¿Qué viaja por el socket y qué nunca sale de cada extremo?

RSA-OAEP • AES-256-GCM • Handshake • Nonce • Authentication tag

Repositorio: `https://github.com/Adriel23456/SecureChat_Workshop`

PROFESOR

MSc. Andrés F. Vargas Rivera

INTEGRANTES — TRABAJO EN PAREJAS

Nombre	Carné
Adriel S. Chaves Salazar	2021031465
Henry D. Nuñez Pérez	2022089224

II Semestre, 2026 • [Fecha de entrega] • Cartago, Costa Rica

Índice

1 Introducción y objetivo **1**
- 1.1 Alcance . . . 1
- 1.2 Entorno de desarrollo . . . 1

2 Arquitectura general **1**
- 2.1 Organización del código . . . 1
- 2.2 Concurrencia . . . 2

3 Material criptográfico **2**
- 3.1 Par de llaves asimétricas . . . 2
- 3.2 Clave de sesión . . . 2
- 3.3 Qué es público, qué es secreto y qué viaja por la red . . . 3

4 Establecimiento de la sesión (handshake) **3**
- 4.1 Mensajes del handshake . . . 4
- 4.2 Confirmación de la sesión . . . 4
- 4.3 Manejo de errores en el handshake . . . 4

5 Protección de los mensajes **4**
- 5.1 AES-256-GCM . . . 4
- 5.2 Generación del nonce . . . 4
- 5.3 Verificación en el receptor . . . 5
- 5.4 Protección contra repetición . . . 5

6 Formato de los mensajes transmitidos **5**

7 Conversación bidireccional y cierre de sesión **5**
- 7.1 Flujo de un mensaje . . . 5
- 7.2 Presentación en consola . . . 6
- 7.3 Cierre controlado . . . 6

8 Detección de manipulación **6**

9 Análisis de seguridad **6**
- 9.1 Modelo de adversario . . . 6
- 9.2 Cumplimiento de los requisitos mínimos . . . 6
- 9.3 Limitaciones conocidas . . . 7

10 Preguntas de cierre **7**

11 Evidencias de la demostración **8**

Referencias **9**

A Ejecución y estructura del repositorio **10**

B Declaración de uso de IA **10**

1. Introducción y objetivo

SecureChat es una aplicación de consola que mantiene una conversación bidireccional entre dos procesos que se ejecutan en la misma computadora, conectados por un socket TCP sobre `localhost`. La criptografía **asimétrica** se usa para establecer la sesión y la criptografía **simétrica autenticada** protege cada mensaje.

[**PENDIENTE:** *Resumen de la solución final en un párrafo: lenguaje, biblioteca, algoritmos, tamaños de llave y resultado de la demostración.*]

1.1. Alcance

[**PENDIENTE:** *Qué cubre y qué no cubre la solución (por ejemplo, autenticación de la llave pública, persistencia, más de dos participantes).*]

1.2. Entorno de desarrollo

Tabla 1.1: Entorno de desarrollo y demostración

Elemento	Detalle
Sistema operativo	Windows [PEND.]
Lenguaje	Python [PEND.]
Biblioteca criptográfica	`cryptography` [PEND.]
Comunicación	Sockets TCP/IP sobre `127.0.0.1`, puerto [PEND.]
Repositorio	`https://github.com/Adriel23456/SecureChat_Workshop`

2. Arquitectura general

El mismo programa se ejecuta dos veces, una instancia por participante (figura 2.1). Bob espera conexiones y Alice inicia la conexión.

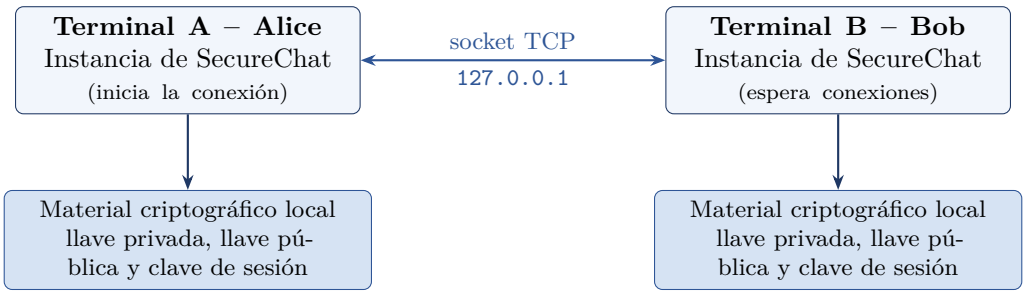


Figura 2.1: Arquitectura de SecureChat: dos instancias del mismo programa conectadas por TCP en la misma computadora.

2.1. Organización del código

[**PENDIENTE:** *Módulos finales y responsabilidad de cada uno (red, framing, llaves, handshake, cifrado, interfaz de consola).*]

2.2. Concurrencia

[**PENDIENTE:** *Cómo se reciben mensajes mientras el usuario escribe (hilo receptor y entrada de teclado) y por qué en Windows no se usa* `select()` *sobre* `stdin`.]

3. Material criptográfico

3.1. Par de llaves asimétricas

Tabla 3.1: Parámetros del par de llaves asimétricas

Parámetro	**Valor**
Algoritmo	RSA
Tamaño del módulo	**[PEND.]** bits (mínimo 2048)
Exponente público	**[PEND.]**
Relleno (padding)	OAEP, hash **[PEND.]**, MGF1 con **[PEND.]**
Formato de la llave privada	**[PEND.]**
Formato de la llave pública	**[PEND.]**
Protección de la llave privada en disco	**[PEND.]**
Distribución de la llave pública	**[PEND.]**

[**PENDIENTE:** *Por qué la llave pública puede compartirse, por qué la privada debe permanecer local y por qué la pública no reemplaza a la privada.*]

3.2. Clave de sesión

[**PENDIENTE:** *Cómo y cuándo se genera la clave de sesión (256 bits, generador aleatorio criptográfico, una nueva por sesión, nunca fija en el código) y cuánto vive.*]

3.3. Qué es público, qué es secreto y qué viaja por la red

Tabla 3.2: Clasificación del material y de los datos del protocolo

Elemento	Dónde vive	¿Viaja por el socket?	Nivel
Llave privada (Alice, Bob)	Disco local de su dueño	Nunca	**Secreto**
Llave pública	Disco local / handshake	**[PEND.]**	**Público**
Clave de sesión	Memoria de ambos procesos	Solo cifrada con RSA-OAEP	**Secreto**
Plaintext del usuario	Solo en la terminal local	Nunca (tras el handshake)	**Secreto**
Nonce	Generado por mensaje	Sí, junto al ciphertext	**Público**
Ciphertext	En tránsito	Sí	**Público**
Authentication tag	En tránsito	Sí	**Público**
Metadatos (puertos, tamaños, horas)	Encabezados TCP/IP	Sí	**Público**

4. Establecimiento de la sesión (handshake)

El handshake se completa antes de permitir cualquier mensaje de usuario (figura 4.1).

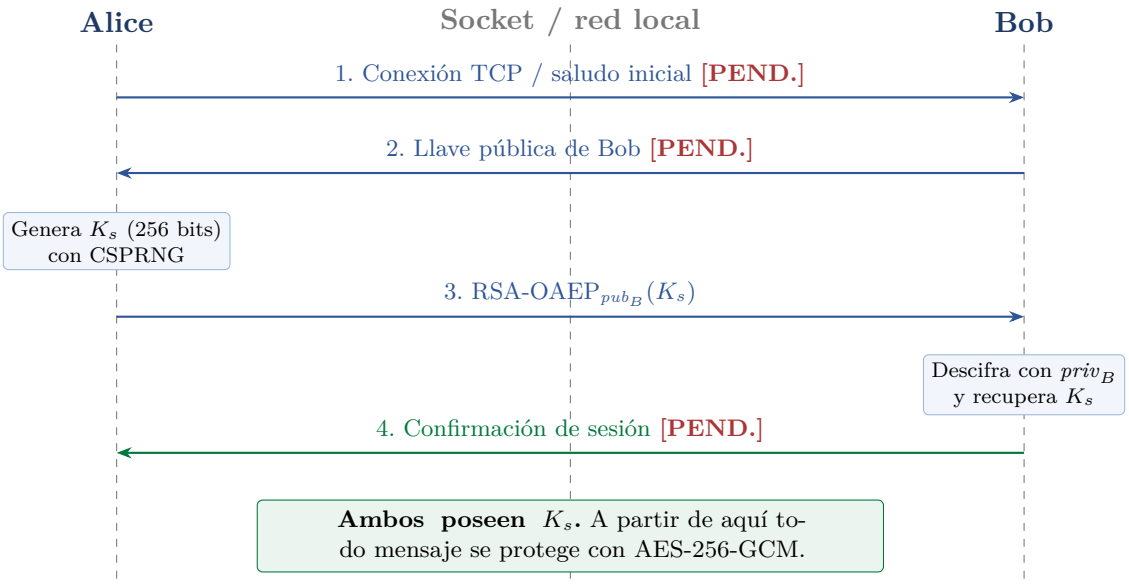


Figura 4.1: Secuencia del handshake (versión preliminar; se actualizará con los mensajes reales).

4.1. Mensajes del handshake

Tabla 4.1: Mensajes del handshake

#	Emisor	Tipo	Contenido y protección
1	Alice	**[PEND.]**	**[PEND.]**
2	Bob	**[PEND.]**	**[PEND.]**
3	Alice	**[PEND.]**	**[PEND.]**
4	Bob	**[PEND.]**	**[PEND.]**

4.2. Confirmación de la sesión

[**PENDIENTE:** *Cómo se prueba que ambos extremos tienen la misma clave de sesión y qué se muestra en pantalla al establecerse la sesión segura.*]

4.3. Manejo de errores en el handshake

[**PENDIENTE:** *Qué ocurre si la llave pública es inválida, el descifrado falla, la confirmación no coincide o se agota el tiempo. Nunca se continúa sin cifrado.*]

Decisión pendiente. Autenticación de la llave pública de Bob. Sin verificarla, un atacante activo en el medio podría sustituirla. [**PENDIENTE:** *Decisión adoptada (por ejemplo, verificación de la huella SHA-256 fuera de banda) y justificación.*]

5. Protección de los mensajes

5.1. AES-256-GCM

Tabla 5.1: Parámetros del cifrado simétrico autenticado

Parámetro	Valor
Algoritmo	AES-256-GCM (cifrado autenticado, AEAD)
Tamaño de clave	256 bits
Tamaño del nonce	96 bits
Tamaño del tag	128 bits
Datos asociados (AAD)	**[PEND.]**
Codificación del texto	**[PEND.]**

5.2. Generación del nonce

[**PENDIENTE:** *Estrategia adoptada (contador por dirección o aleatorio), cómo se evita la repetición con la misma clave considerando que ambos sentidos comparten la clave de sesión, y qué pasa si el contador se agota.*]

5.3. Verificación en el receptor

[**PENDIENTE:** *El receptor verifica el tag antes de aceptar el plaintext; si falla, el mensaje se rechaza y no se muestra contenido.*]

5.4. Protección contra repetición

[**PENDIENTE:** *Uso de números de secuencia autenticados o justificación si no se implementa.*]

6. Formato de los mensajes transmitidos

TCP entrega un flujo de bytes sin límites de mensaje, por lo que cada mensaje se delimita con un prefijo de longitud. [**PENDIENTE:** *Confirmar.*]

Tabla 6.1: Estructura de una trama cifrada (propuesta preliminar)

Campo	Tamaño	Descripción
Longitud	[PEND.]	Tamaño del resto de la trama
Tipo	[PEND.]	Tipo de mensaje
Secuencia	[PEND.]	Número de mensaje por dirección
Nonce	12 bytes	Único por cifrado con la misma clave
Ciphertext	variable	Mensaje cifrado
Tag	16 bytes	Authentication tag de GCM

[**PENDIENTE:** *Diagrama de la trama en bytes y ejemplo real capturado.*]

Codificación no es cifrado. La representación en hexadecimal o Base64 se usa solo para mostrar o transportar bytes. No ofrece confidencialidad.

7. Conversación bidireccional y cierre de sesión

7.1. Flujo de un mensaje

Mensaje legible → AES-GCM cifra y autentica → ciphertext en tránsito → verificación y descifrado → mensaje recuperado.

[**PENDIENTE:** *Diagrama de secuencia de un mensaje en cada sentido.*]

7.2. Presentación en consola

Tabla 7.1: Distinción visual en la consola

Etiqueta	Significado
[PEND.]	Mensaje local (plaintext antes de cifrar)
[PEND.]	Datos cifrados enviados
[PEND.]	Datos cifrados recibidos
[PEND.]	Mensaje recuperado tras verificar el tag
[PEND.]	Error o mensaje rechazado

7.3. Cierre controlado

[**PENDIENTE:** *Comando de salida, mensaje de cierre cifrado y comportamiento del otro extremo.*]

8. Detección de manipulación

[**PENDIENTE:** *Mecanismo de la prueba controlada (alteración en el emisor o proxy intermedio), qué byte se altera y por qué.*]

[**PENDIENTE:** *Resultado: el receptor detecta el tag inválido, rechaza el mensaje y no muestra plaintext. Política tras el rechazo (descartar o cerrar la sesión).*]

IMAGEN NO ENCONTRADA
05_manipulacion.png

Figura 8.1: Rechazo de un ciphertext alterado.

9. Análisis de seguridad

9.1. Modelo de adversario

[**PENDIENTE:** *Adversario pasivo (escucha el socket) y activo (modifica, repite o inyecta), en la línea del modelo Dolev-Yao visto en clase.*]

9.2. Cumplimiento de los requisitos mínimos

Tabla 9.1: Requisitos mínimos del taller y su cumplimiento

	Requisito	Cómo se cumple	Evidencia
✓	Python 3 y sockets TCP/IP	[PEND.]	[PEND.]

Continúa en la página siguiente

Tabla 9.1 – viene de la página anterior

	Requisito	Cómo se cumple	Evidencia
✓	Biblioteca criptográfica reconocida	[PEND.]	[PEND.]
✓	Handshake basado en criptografía asimétrica	[PEND.]	[PEND.]
✓	Clave de sesión dinámica, no fija en el código	[PEND.]	[PEND.]
✓	Ninguna llave privada viaja por el socket	[PEND.]	[PEND.]
✓	Mensajes cifrados con esquema autenticado	[PEND.]	[PEND.]
✓	Nonce correcto y no repetido	[PEND.]	[PEND.]
✓	Rechazo de tags inválidos	[PEND.]	[PEND.]
✓	Evidencia de plaintext, ciphertext enviado y recibido, y plaintext recuperado	[PEND.]	[PEND.]
×	Base64 o hexadecimal no sustituyen al cifrado	[PEND.]	[PEND.]
×	No se usa ECB	[PEND.]	[PEND.]

9.3. Limitaciones conocidas

[**PENDIENTE:** *Por ejemplo: autenticación de la llave pública, ausencia de secreto hacia adelante (forward secrecy) al transportar la clave con RSA, análisis de tráfico (tamaños y tiempos visibles), y la imposibilidad de borrar con certeza secretos de la memoria en Python.*]

10. Preguntas de cierre

1. **¿Qué problema resuelve la criptografía asimétrica durante el handshake?**
   [**PENDIENTE:** *Respuesta.*]
2. **¿Por qué la conversación utiliza criptografía simétrica después del handshake?**
   [**PENDIENTE:** *Respuesta.*]
3. **¿Qué información puede observar un tercero que capture los paquetes de la conversación?**
   [**PENDIENTE:** *Respuesta.*]
4. **¿Por qué el nonce puede viajar junto al ciphertext y por qué no debe repetirse?**
   [**PENDIENTE:** *Respuesta.*]
5. **¿Qué función cumple el authentication tag?**
   [**PENDIENTE:** *Respuesta.*]
6. **¿Qué ocurriría si la clave de sesión se transmitiera sin protección?**
   [**PENDIENTE:** *Respuesta.*]
7. **¿Qué ocurriría si un atacante obtiene una de las claves privadas?**

[**PENDIENTE:** *Respuesta.*]

11. Evidencias de la demostración

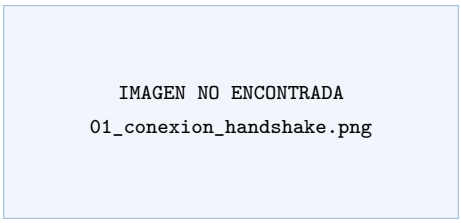


Figura 11.1: Dos instancias en ejecución y handshake completado.

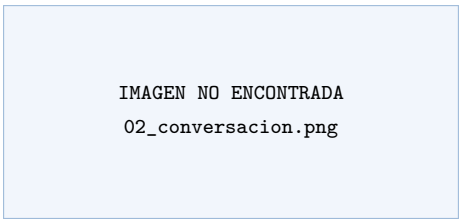


Figura 11.2: Conversación con al menos tres mensajes por sentido.

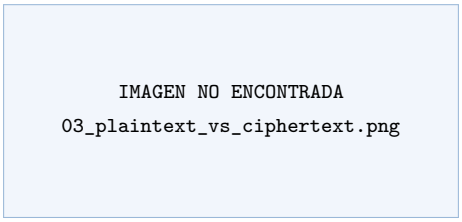


Figura 11.3: Comparación entre el mensaje legible y el ciphertext transmitido.

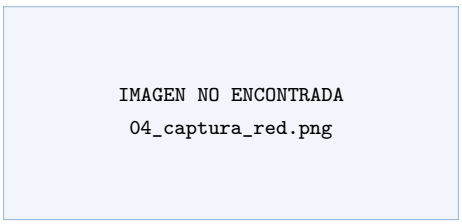


Figura 11.4: (Opcional) Captura del tráfico de `localhost`.

[**PENDIENTE:** *Enlace al video de demostración.*]

Referencias

A. Ejecución y estructura del repositorio

[**PENDIENTE:** *Estructura final del repositorio y comandos para instalar, generar llaves e iniciar cada instancia en Windows (resumen del README).*]

B. Declaración de uso de IA

[**PENDIENTE:** *Herramientas usadas, para qué y cómo se verificó el resultado.*]